complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() updating external systems

filter() null checks + foreach() updating external systems

filter() comparisons + foreach() updating external systems

filter() method calls returning boolean + foreach() updating external systems

distinct() rely on equals() / hashcode() + foreach() updating external systems

distinct() immutable objects + foreach() updating external systems

map() transform objects + foreach() updating external systems

map() create new objects + foreach() updating external systems

map() call pure functions + foreach() updating external systems

flatmap() return stream + foreach() updating external systems

flatmap() split data + foreach() updating external systems

flatmap() expand collections + foreach() updating external systems

sorted() natural ordering + foreach() updating external systems

sorted() custom comparator + foreach() updating external systems

sorted() multi-field compare + foreach() updating external systems

peek() logging + foreach() updating external systems

peek() debugging + foreach() updating external systems

peek() metrics + foreach() updating external systems

Below are **complete, standalone Java class examples** showing **side-effects with forEach()** where the stream **updates an external system** (simulated as DB / cache / message queue / API).

 Important concept reminder (intentional here):
Using forEach() to update external systems is a **side-effect** and must be done **only at the terminal operation**. These examples are **educational and realistic**, not advocating misuse.

All examples:

- Java **8+**
- One intermediate concept per class
- External system = simulated service (ExternalSystem.update(...))

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

---

1 filter() boolean checks + forEach() Updating external systems

```
import java.util.*;

public class FilterBooleanUpdateExternal {
  public static void main(String[] args) {
    List<Integer> orders = List.of(100, 150, 200);

    orders.stream()
        .filter(o -> o >= 150)
        .forEach(o -> ExternalSystem.update("Approved order: " + o));
  }
}
```

---

2 filter() null checks + forEach() Updating external systems

```
import java.util.*;

public class FilterNullUpdateExternal {
   public static void main(String[] args) {
      List<String> users = Arrays.asList("Anurag", null, "Amit");

      users.stream()
            .filter(Objects::nonNull)
            .forEach(u -> ExternalSystem.update("User synced: " + u));
   }
}
```

---

3 filter() comparisons + forEach() Updating external systems

```
import java.util.*;

public class FilterComparisonUpdateExternal {
   public static void main(String[] args) {
      List<Integer> balances = List.of(500, 2000, 7000);

      balances.stream()
            .filter(b -> b > 1000)
            .forEach(b -> ExternalSystem.update("Premium balance: " + b));
   }
}
```

---

4 filter() method call returning boolean + forEach() Updating external systems

```
import java.util.*;

public class FilterMethodUpdateExternal {

   static class Account {
      String id;
      boolean active;

      Account(String id, boolean active) {
         this.id = id;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<Account> accounts = List.of(
            new Account("A1", true),
            new Account("A2", false)
      );

      accounts.stream()
            .filter(Account::isActive)
            .forEach(a -> ExternalSystem.update("Activated account: " + a.id));
   }
}
```

---

5 distinct() equals/hashCode + forEach() Updating external systems

```
import java.util.*;

public class DistinctEqualsUpdateExternal {

   static class Customer {
      String email;

      Customer(String email) { this.email = email; }

      @Override
      public boolean equals(Object o) {
         return o instanceof Customer c && email.equals(c.email);
      }

      @Override
      public int hashCode() {
         return Objects.hash(email);
      }
   }

   public static void main(String[] args) {
      List<Customer> customers = List.of(
            new Customer("a@mail.com"),
            new Customer("a@mail.com"),
            new Customer("b@mail.com")
      );

      customers.stream()
            .distinct()
            .forEach(c -> ExternalSystem.update("Customer saved: " + c.email));
   }
}
```

---

6 distinct() immutable objects + forEach() Updating external systems

```
import java.util.*;

public class DistinctImmutableUpdateExternal {
   public static void main(String[] args) {
      List<String> tags = List.of("java", "java", "spring");

      tags.stream()
            .distinct()
            .forEach(t -> ExternalSystem.update("Tag indexed: " + t));
   }
}
```

---

7 map() transform objects + forEach() Updating external systems

```
import java.util.*;

public class MapTransformUpdateExternal {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      names.stream()
            .map(String::toUpperCase)
            .forEach(n -> ExternalSystem.update("Normalized name: " + n));
   }
}
```

---

8 map() create new objects + forEach() Updating external systems

```
import java.util.*;

public class MapNewObjectUpdateExternal {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      names.stream()
            .map(UserDTO::new)
            .forEach(dto -> ExternalSystem.update("DTO persisted: " + dto.name));
   }
}
```

9 map() call pure functions + forEach() Updating external systems

```
import java.util.*;

public class MapPureFunctionUpdateExternal {

   static int calculateScore(int n) {
      return n * 10;
   }

   public static void main(String[] args) {
      List<Integer> inputs = List.of(1, 2, 3);

      inputs.stream()
            .map(MapPureFunctionUpdateExternal::calculateScore)
            .forEach(score -> ExternalSystem.update("Score sent: " + score));
   }
}
```

10 flatMap() return Stream + forEach() Updating external systems

```
import java.util.*;

public class FlatMapStreamUpdateExternal {
   public static void main(String[] args) {
      List<List<String>> batches = List.of(
            List.of("A", "B"),
            List.of("C")
      );

      batches.stream()
            .flatMap(List::stream)
            .forEach(v -> ExternalSystem.update("Batch item: " + v));
   }
}
```

11 flatMap() split data + forEach() Updating external systems

```
import java.util.*;

public class FlatMapSplitUpdateExternal {
   public static void main(String[] args) {
      List<String> lines = List.of("Java Stream", "Flat Map");

      lines.stream()
            .flatMap(l -> Arrays.stream(l.split(" ")))
            .forEach(word -> ExternalSystem.update("Word indexed: " + word));
   }
}
```

---

12 flatMap() expand collections + forEach() Updating external systems

```
import java.util.*;

public class FlatMapExpandUpdateExternal {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      );

      data.values().stream()
            .flatMap(Collection::stream)
            .forEach(v -> ExternalSystem.update("Value synced: " + v));
   }
}
```

---

13 sorted() natural ordering + forEach() Updating external systems

```
import java.util.*;

public class SortedNaturalUpdateExternal {
   public static void main(String[] args) {
      List<Integer> priorities = List.of(3, 1, 2);

      priorities.stream()
            .sorted()
            .forEach(p -> ExternalSystem.update("Processed priority: " + p));
   }
}
```

---

14 sorted() custom comparator + forEach() Updating external systems

```
import java.util.*;

public class SortedCustomUpdateExternal {
   public static void main(String[] args) {
      List<String> jobs = List.of("LOW", "HIGH", "MEDIUM");

      jobs.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(j -> ExternalSystem.update("Job executed: " + j));
   }
}
```

---

15 sorted() multi-field compare + forEach() Updating external systems

```
import java.util.*;

public class SortedMultiFieldUpdateExternal {

   static class Task {
      String name;
      int priority;

      Task(String name, int priority) {
         this.name = name;
         this.priority = priority;
      }
   }

   public static void main(String[] args) {
      List<Task> tasks = List.of(
            new Task("Deploy", 2),
            new Task("Build", 1),
            new Task("Test", 2)
      );

      tasks.stream()
            .sorted(Comparator
                  .comparing((Task t) -> t.priority)
                  .thenComparing(t -> t.name))
            .forEach(t ->
                  ExternalSystem.update("Task executed: " + t.name));
   }
}
```

---

16 peek() logging + forEach() Updating external systems

```
import java.util.*;

public class PeekLoggingUpdateExternal {
   public static void main(String[] args) {
      List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek log: " + n))
            .forEach(n -> ExternalSystem.update("Final update: " + n));
   }
}
```

---

17 peek() debugging + forEach() Updating external systems

```
import java.util.*;

public class PeekDebugUpdateExternal {
   public static void main(String[] args) {
      List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug breakpoint hit");
            })
            .forEach(n -> ExternalSystem.update("Processed value: " + n));
   }
}
```

---

18 peek() metrics + forEach() Updating external systems

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsUpdateExternal {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(n -> ExternalSystem.update("Metric sent: " + n));

      ExternalSystem.update("Total processed: " + counter.get());
   }
}
```